

Original papers

Apple orchard production estimation using deep learning strategies: A comparison of tracking-by-detection algorithms

Juan Villacrés^a, Michelle Viscaino^b, José Delpiano^c, Stavros Vougioukas^a,
Fernando Auat Cheein^{b,*}

^a Department of Biological and Agricultural Engineering, University of California, Davis, USA

^b Department of Electronic Engineering, Universidad Técnica Federico Santa María, Valparaíso, Chile

^c Department of Engineering, Universidad de los Andes, Santiago, Chile

ARTICLE INFO

Keywords:

Apple counting
Tracking-by-detection
Faster R-CNN
YoloV5
Multi-object tracking

ABSTRACT

The automated detection and counting of fruit in tree canopies is a key component of yield estimation systems, which are indispensable for the precision management of modern orchards. Detection and counting tasks in agricultural environments are not trivial because of challenges such as characteristics of the tree canopies, occlusion caused by leaves and the lighting conditions, among other factors. With the aim of identifying which algorithm is more suitable for yield estimation, we present a comprehensive comparison of tracking-by-detection algorithms, applied to apple counting. The tracking strategies evaluated were Kalman Filter, Kernelized Correlation Filter, Simple Online Real-Time Tracking, Multi Hypothesis Tracking, and Deep Simple Online Real-Time Tracking. The five tracking algorithms were further assessed on two novel databases constructed for this research in Multiple Object Tracking MOT format. After a sensitivity analysis of the trackers, the results show that the most robust approach is the Multiple Hypothesis Tracking, followed by the Deep Simple Online Realtime (DeepSORT), with a MOT accuracy of 97.00% and 93.00%, respectively, when having perfect detection. However, in an application case including a deep learning-based detection stage, the DeepSORT tracker obtains the lowest counting error, which on average for all videos is 20.07% and 31.52% when using YoloV5 and Faster R-CNN as detection strategies. Statistically similar results were obtained using the Kalman Filter with a counting error of 20.5% and 31.9% when detecting fruit with YoloV5 and Faster R-CNN.

1. Introduction

Automated yield estimation in fruit or vegetable fields enables better resource management in harvesting, storage and transportation. In the absence of automated yield estimation systems in fruit crops, orchard managers use manual fruit counting by randomly choosing a few samples of tree branches, performing the counting, and extrapolating the results to the entire orchard blocks (Häni et al., 2020). Although this method is simple, it is time-consuming and error-prone up to 66% (Meena et al., 2014). Therefore, developing automated systems for fruit load estimation is a significant challenge in precision agriculture to achieve higher productivity (Meena et al., 2014). Computer vision is the prevailing method to estimate fruit load. One or more cameras are typically carried along the orchard rows, and consecutive frames of the tree canopies are processed to detect and count fruit, while avoiding the same fruit multiple times. The main tasks involved in computer vision-based fruit load estimation include detecting, counting, and

tracking fruit in multiple images. These tasks can be addressed as individual tasks or as a more complex scheme that involves an end-to-end processing pipeline. Regardless of the approach, detection and counting depend on the characteristics of agricultural environments, such as variations in illumination, complex backgrounds, occlusions produced by leaves, branches, or between fruit, camera's distance, and viewing (Tian et al., 2019; Kang and Chen, 2020; Villacrés and Cheein, 2020).

Technological advances in visual sensors and improved computer vision techniques have enabled the development of efficient detection strategies. The detection sensor of choice has been the RGB (red, green and blue) camera, partly because of its low cost and ease of implementation (Koirala et al., 2019a). Early work on fruit detection was based on handcrafted features extracted from images. However, the unstructured conditions of the agricultural environment require robust features that should not need to be redesigned when visual changes

* Corresponding author.

E-mail address: fernando.auat@usm.cl (F. Auat Cheein).

occur in the object to be detected (e.g., changes in fruit ripeness). A detailed description of the platforms, sensors and methods for indirect and direct estimation of orchard yield are presented in He et al. (2022).

In general, detection strategies for yield estimation exhibit better results when supervised learning is used instead of simple image processing with handcrafted features (e.g., fruit color, texture, and shape) (Koirala et al., 2019a). In Roy and Bhaduri (2022), fruit detection in critical conditions such as occluded scenarios is addressed by the fusion of YoloV4 with a DenseNet as a backbone to detect mangoes. The results exhibited a mean average precision (mAP) of 96.20% and an F1-score of 93.61%, showing higher performance than the state-of-the-art YoloV4 detector (Roy and Bhaduri, 2022). Another custom detection architecture for fruit counting is MangoYolo (Koirala et al., 2019b). MangoYolo utilizes the YoloV3 and YoloV2(tiny) architectures to achieve fast and accurate detections. MangoYolo outperformed YoloV3, YoloV2(tiny), Faster R-CNN (VGG) and Faster R-CNN (ZF) in detection speed and accuracy, as well as memory usage. Given the fast detection speed of Yolo-based detectors, these architectures can be used in computationally constrained applications (e.g., embedded systems) and under the requirement of real-time execution. For example, Zhang et al. (2022) published a study in which an embedded system (Jetson Nano) was used to identify strawberries in real-time. The architectures employed were YoloV4(tiny) and a modified version of itself—the authors dubbed the architecture RTSD-Net (Zhang et al., 2022). In the case of multi-class detection (i.e., fruit varieties), Vasconez et al. (2020) presented the relative error for the count of avocados, apples, and lemons. In this work, the authors obtained a relative error – the ratio of the absolute error in the estimate and the actual value – of 7%, 13%, and 20% using Faster R-CNN; and 18%, 10%, and 25% using Single Shot Detector (SSD) for avocados, apple, and lemons, respectively. Additionally, Wan and Goudos (2020) reported that a modification of Faster R-CNN achieved a mAP above 91%, which is better than YoloV3, in detecting apples, oranges and mangoes.

Although the detection stage in the processing pipeline for automated fruit load estimation has seen rapid advances culminating in deep learning strategies, the counting part is still under development, with several challenges to overcome. One of the main drawbacks of image-based fruit counting lies in the fact that fruit yield estimation requires a set of images that, if not handled properly, may overlap, resulting in yield overestimation. Therefore, it is necessary to develop an accurate solution for processing image sequences, such as the ones found in videos. A viable solution is to use a sequence of images that do not overlap. Such non-overlapping prevents the fruit from being detected multiple times because it is only visible in a single frame (Villacrés and Cheein, 2020). As a result, the total fruit load corresponds to the number of fruits detected in the non-overlapping image sequence. In particular, this non-overlap is achieved by moving the camera at a constant speed during acquisition and selecting an appropriate subsampling frequency (Hung et al., 2015). In this approach, errors in sampling frequency selection or non-constant speed during data acquisition can lead to overlaps between images and subsequently, counting some fruit multiple times. A disadvantage of non-overlapping images is that not all fruit are likely to be visible from the camera's perspective. Another approach being explored in recent years is to assign a unique identifier to each fruit/vegetable along the image sequence. This approach is known as multiple object tracking (MOT) and is predominantly used for pedestrian tracking. Several databases exist for pedestrian MOT, such as MOT15, MOT16, and MOT17. In these databases, the ground truth for each pedestrian consists of a bounding box and a unique identifier (ID) (Dendorfer et al., 2021). Such databases can be obtained from stationary surveillance cameras or moving cameras. Since the fruit in canopies are essentially static, creating and processing videos acquired with a moving camera is necessary. To the best of our knowledge, there are no public databases to compare the performance of MOT algorithms on fruit. Therefore, it becomes difficult to evaluate the performance of MOT in agriculture. The main contributions of this paper are:

- The creation of a public dataset for multiple object tracking for apples.
- The analysis of the sensitivity of several algorithms for multiple object tracking in an agricultural environment.
- The evaluation of the feasibility of MOT algorithms in a real application for fruit counting.

This work is organized as follows: Section 2 presents the algorithms used in the context of multiple object tracking to count fruit and how missing fruit was handled in the tracking process; Section 3 includes the databases used, as well the tracking algorithms. Section 4 presents the results obtained in the sensitivity analysis and an application task; Section 5 depicts the discussion of the results, and finally, Section 6 includes the conclusions obtained in this research.

2. Related work

With the advancement of image processing techniques – particularly frameworks based on deep neural networks –, the detection and counting of fruit in images has improved. However, it is necessary to consider other challenges, such as fruit recounting in a sequence of images – i.e., a video. Fruit tracking is conducted through single-view or multiple-view images – the latter aim to tackle occlusion caused by branches, leaves, and even other fruit. Below, we list recent works employing tracking strategies for fruit counting in image sequences.

2.1. Single-view cameras

Hamuda et al. (2018) proposes the use of a Kalman filter (KF) to estimate the centers of the bounding boxes of cauliflower plants, and Hungarian algorithm (HA) for data association in consequent images. The reported MOT results outperformed those obtained with optical flow, 99.34% vs 97.28% recall, respectively. A tracker is removed if the object in the frame is missed more times than a defined threshold; in this case, Hamuda et al. (2018) defined it as 10. In addition, if the object is not detected and with thresholds below 10, tracking was performed using the Kalman filter predictions. The Kalman filter was also applied by Wang et al. (2019) to predict the states of bounding boxes obtained with Yolo for mango detection. The Hungarian algorithm was used to perform tracking using the multiple-to-one assignment. The authors found in practice that the Kalman filter prediction model was effective if there were at least four contiguous measurements. In the absence of adjacent measures, the velocity of neighboring fruit was used, and if there were no “stable” neighbors, a constant velocity criterion was applied. Once a “stable” model was obtained, a prediction of up to 15 frames from the last fruit detection was performed. One of the noteworthy drawbacks of using the Kalman Filter is that most papers generally consider a model in which the velocity is constant over a time interval. Such a model could be affected in case of vibrations during data acquisition. On the other hand, the Kalman Filter could be suitable in scenarios where there are occlusions because it only depends on the object's motion; this means that the Kalman Filter predictions could be used to estimate the object's position in the following frames. However, the predictions may not be as reliable if there are not enough new measurements to update the Kalman filter. Jarvinen et al. (2018) obtained a mean error of 11% in apple counting when using optical flow as the tracking strategy. The authors used Faster R-CNN architecture to detect apples, and the intersection over the union criterion for matching detections. Optical flow was used to estimate pixel motion instead of object motion. Once the translation of a frame at instant t and time $t + 1$ was determined, the fruit detections were translated and associated with the new detections. In the scenarios where apples were missing (i.e., not detected with Faster R-CNN), the bounding boxes were translated considering the displacement obtained with optical flow. A disadvantage of using Optical Flow in tracking tasks is that Optical Flow cannot adequately deal with situations where occlusions are present.

Such translations of the detections were repeated until the bounding boxes reached the boundary of the image. Similarly, Gan et al. (2018) used the Faster R-CNN to detect citrus in thermal imaging. They applied optical flow to avoid overcounting fruit, with an average accuracy of 87.2%. In Vasconez et al. (2020), a comprehensive evaluation of fruit detection and counting was conducted; bayesian filtering allowed multiple fruit (apples, lemons, and avocado) tracking across 2, 15, 30, 60, and 100 frames. Other work presented by Liu et al. (2018) used the Hungarian algorithm to track apples and oranges in image sequences. Instead of using Euclidean distance as the objective function, they used a Kalman filter-based cost function improved with optical flow. In addition, the authors corrected the number of counted fruit using the 3D map based on structure from motion (SfM). Fruit counting was performed only when a predefined age was exceeded, and such fruit was added to the final count at the time it disappeared. This means that if a fruit reappears, it is possible that the count will be increased. In Halstead et al. (2018), an empirical approach was proposed for tracking sweet peppers, which employed the intersection over union (IoU) tracking strategy and the Faster R-CNN detector. The errors in tracking green peppers, mixed peppers and red peppers were 11.2%, 1.8% and 0.9%, respectively. The tracker was considered inactive if it was not detected for three frames and was therefore removed. The counting of apples and oranges was addressed by Xia et al. (2022), Gao et al. (2022), where the fruit detection is performed with the CenterNet architecture. The work employed the patches detected by the CenterNet, resized them to a fixed size, and used them as inputs to a matching network. Such a matching network concatenates two patches and obtains a similarity metric. However, this process is expensive because it must compare all patches against each other in a brute-force manner. The optimal association of the detected fruit along contiguous frames was performed with the Hungarian algorithm (i.e., the Kuhn–Munkres algorithm). The results presented showed a coefficient of determination of 0.937 and 0.9562 for apples and oranges, respectively. Another fruit-tracking approach is the kernelized correlation filter (KCF), which outperforms the Kalman filter-based tracker when tracking green fruits (Zhang et al., 2020). In particular, the KCF achieved an average tracking success of 94.07%, 87.35%, and 61.46% when tracking one, two, and three apples, respectively. The Kalman and Kernelized Correlation filters are strategies that achieve real-time implementation. However, the KCF was the faster among the two. A disadvantage of the Kalman filter is that when oscillations occur, it is possible that the object appears and disappears, which is reflected in the Kalman filter not being able to maintain the previous tracking state. In addition to using RGB images, representations such as the NDVI index were used to classify tomatoes by maturity stage (Chen and Lin, 2020). And subsequently, track unique identifiers to avoid repetition in tomato counting using Simple Online and Realtime Tracking (SORT) and DeepSORT. In summary, DeepSORT slightly outperformed the SORT tracker in terms of Multiple Object Tracking Precision (MOTP). It is important to note that the advantage of SORT over DeepSORT is the speed of execution of the algorithm, mainly because SORT uses only a linear model of the object motion. Whereas, DeepSORT also includes a descriptor of the object's appearance. An alternative for apple counting was presented by Gao et al. (2021), where tree trunks were considered as the tracking target instead of apples. The main reason for tracking the trunk was that it was larger than the fruit and had the same trajectory as the apples. Therefore, the fruit is detected and the movement of the trunk is used to estimate the movement of the fruit. However, a drawback of this approach is that if the trunk is not visible (e.g., cases of occlusion by leaves or fruits if the camera points directly at the fruits), the tracking could not be carried out.

2.2. Multiple-view cameras

As for the multiple view systems, Häni et al. (2020) used two image sequences, the first corresponding to a front view of an orchard row

and the second corresponding to the rear view of the same orchard row. A 3D reconstruction represented as a semi-dense point cloud was obtained using the semantic information of the two sequences. The algorithm for the 3D reconstruction using the two views of the same orchard row is described in Roy et al. (2018). Such a point cloud was projected onto each camera frame, and the counting was performed on the three frames containing the most pixels belonging to apples. Thus, if apples are missing in some frames, they were not recounted because the point cloud is used as a reference. Counting of mangoes using two cameras with opposite side views of the trees was presented by Liu et al. (2019), where the problem was posed as a Multi Hypothesis Tracking task. The two views allowed the creation of two point clouds for each row; therefore, it was necessary to integrate the point clouds to avoid double counting. The authors point out that the centroid of the tree (i.e., the trunk) was used to separate the tree into two parts. Consequently, only fruits on the side closest to the trunk are counted; in this way, the results indicated that overcounting was reduced. Mango detection was achieved with the Faster R-CNN framework, and fruit and trunk tracking was implemented with the Kalman filter and optical flow. Such tracking was developed in two stages; in the second stage, detections missed in the intermediate frames were processed. A total of 18 trees were analyzed, and the per-tree count error mean obtained was 27.8 and 19.8 for using the single-view and multiple-view systems, respectively. Stein et al. (2016) addressed mango counting by employing single-view, dual-view, and multi-view methods. The detection of mangoes was carried out using a faster R-CNN framework approached the detection of mangoes using a faster R-CNN framework. They then used GPS/INS information to obtain the absolute and relative position of the camera; the mango tracking was addressed using epipolar geometry and the Hungarian algorithm. Detections with no association were eliminated as they were considered false positives, and the results suggest that the proposed multi-view methodology counted even fruit that were hidden from two opposing dual views. The results of the multi-view method indicated that there is a 1.36% counting error for individual trees.

The works mentioned above addressed different tracking strategies for fruit counting. However, there is no benchmark for comparison among them. Each approach performs tracking under various conditions such as illumination, duration of the image sequence, degree of occlusion, number of fruit, and evaluation metrics. One reason is the lack of a public database to evaluate the performance of the trackers; instead, efforts have focused on developing databases for detection. Additionally, the works have employed private databases or even evaluated their experiments under ideal laboratory conditions (i.e., controlled conditions of illumination or low or no occlusion).

In this work, we evaluate five state-of-the-art trackers, with application in agriculture: Kalman filter, kernelized correlation filter (KFC), multi hypothesis tracking (MHT), simple online real-time tracking (SORT), and DeepSORT. We also built a database for performance evaluation, which it can be used as a benchmark for future tracking algorithms. Since our approach uses the tracking-by-detection framework, we include a sensitivity analysis to evaluate the robustness of the trackers to detector performance.

3. Materials and methods

The general architecture employed in this work is presented in Fig. 1. Briefly, the input consists of a sequence of images acquired with a handheld camera facing the apple trees. These images pass through a generic fruit detection system that determines the apples' locations in the image frame. Next, bounding boxes, generated in the previous part, are used to track the images with a set of tracking algorithms. Finally, the output corresponds to the apples tracked across the frames, including the unique identifier.

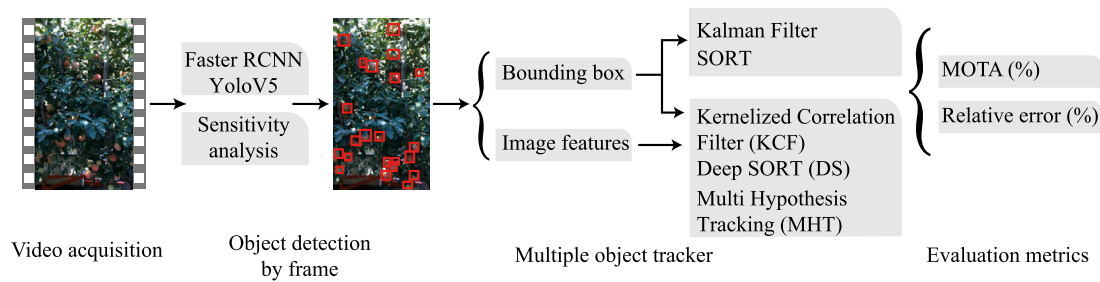


Fig. 1. General tracking scheme. The videos are acquired then the apples are identified by each image. Finally, the object tracker uses the bounding boxes and also image features.

Table 1
Summary of databases used for the object detector pipelines in the case study.

Video name	Resolution	FPS	# Frames	# Object instances	# Apples
T1	1920 × 1080	30	1290	48,111	939
T2	1920 × 1080	30	1170	45,855	849
T3	1280 × 720	120	4589	114,850	624
T4	1280 × 720	120	4445	159,574	1033
T5	1280 × 720	120	2388	68,722	519
T6	1280 × 720	120	3306	129,588	674
T7	1920 × 1080	30	1459	55,443	1693
T8	1920 × 1080	30	1841	58,145	1782
T9	1280 × 720	120	5523	176,236	1092
Total	–	–	26,011	856,524	8845

3.1. Databases description

The databases that were used in this study are listed below. The first database, which contains videos labeled in MOT format, was acquired for this research. The second is a group of public databases for apple detection.

3.1.1. Tracking data sets

The two tracking data sets were collected in 2016 and 2021, respectively, from an orchard located in California, USA (38° 03'55.4"N 121° 11'45.4"W). The images were obtained with a handheld camera in vertical orientation while a person walks down the row. The videos obtained in 2016 were acquired with an iPhone 6s plus, a 12-megapixel camera, an aperture of f/2.2, and optical stabilization. On the other hand, the 2021 videos were taken with an iPhone 13, which has a 12-megapixel camera, three lenses with apertures of f/2.8, f/1.8 and f/1.5, and optical image stabilization. The databases used to assess the multiple object trackers are presented below.

- The first apple tracking database (2016) consists of nine videos. Table 1 shows the main characteristic such as the resolution, frames per second, and the number of frames of each video. As can be seen, the number of frames is different from each video due to the videos having different duration.
- The second apple tracking database (2021) is composed of seven videos with a resolution of 1920 × 1080, 30 fps, and 1 min per video. As a result, a total of 1800 frames per video are obtained. Fig. 2(a)–2(g), present one snapshot of each video. In addition, Fig. 2(h) represents the apple location distribution with respect the camera frame. The majority of the apples are clustered in the middle to the top half of the photos, as can be observed in Fig. 2(h). Generally, the lower part corresponds to the tree trunk or ground. On the other hand, Fig. 3 is a violin plot of the width, height, and area of the labeled bounding boxes. The violin plot generally represents the median, interquartile ranges, and data distribution.

The speed at which the videos were acquired was not recorded accurately (e.g., with an RTK). However, Fig. 4 presents an analysis of the velocity using optical flow with the Lucas–Kanade method. This figure depicts the mean and variance of the translation between frames

expressed as the magnitude of optical flow. It is important to note that in the worst case (i.e., video 2 acquired in 2021), the variance does not exceed 14% of the mean velocity.

All videos were labeled in MOT format using the Computer Vision Annotation Tool (CVAT).¹ In general, the MOT format, as given by the CVAT tool is: [frame, ID, x, y, w, h, Cf, C, v] where: frame, is the number of the frame; ID, is the unique identifier across the entire video sequence; x, y, w and h are the bounding box coordinates left, top, width, and height, respectively; Cf, is the confidence; C is the class identifier; and v, is the visibility. The class identifier in all the databases is 1, since there is only one class. In addition, considering that the data represents the ground-truth, the confidence and visibility are set as 1. Sometimes, if the apple is greenish, it could be confused with the leaves. However, unlike image labeling, video provides the advantage that eventually, the apple will be visible in a particular frame. With that knowledge, human labeling can return to previous frames to draw the corresponding bounding box.

3.1.2. Detection datasets

In Section 4.2, there is an application test for apple counting. In that case, an object detector must provide the bounding boxes as input to the tracking algorithms. For the application to be as realistic as possible, the detection model must generalize to images not seen in training and validation. We employ three public databases to train the object detector to ensure a complete de-correlation of the detection and tracking datasets. The three public databases containing RGB images of apples are presented below:

- The MinneApple dataset was collected between 2015 and 2016 at the University of Minnesota Horticultural Research Center. This dataset consists of 1000 images with a resolution of 720 × 1080 pixels and more than 41,000 apples. The number of object instances (apples) per image ranges from 1 to 120, with 41.2 apples on average. The format of ground truth is given in the form of masks (Hani et al., 2020).

¹ <https://github.com/openvinotoolkit/cvat>

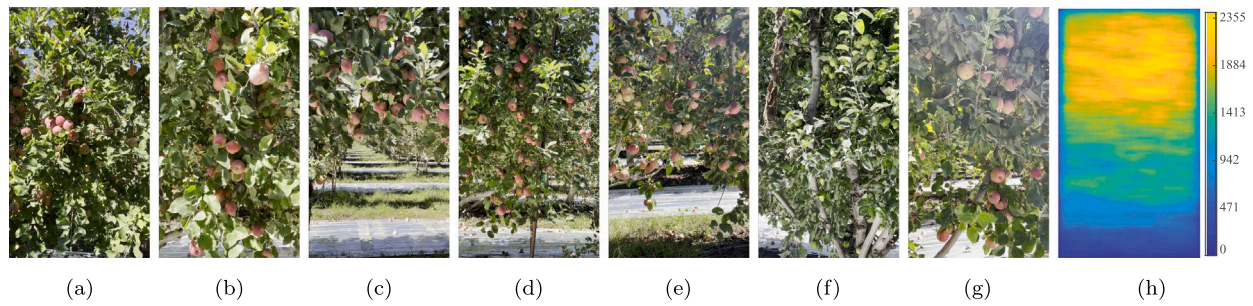


Fig. 2. Snapshots of videos taken in 2021 and subsequently used to perform the sensitivity analysis. (a–g) Depicts one frame from each video annotated from 1 to 7, and (h) a summary of the distribution of apples in the seven video tracks. The more yellow the color tone, the more apples were observed throughout the video. Frames per second of the videos correspond to 30 fps.

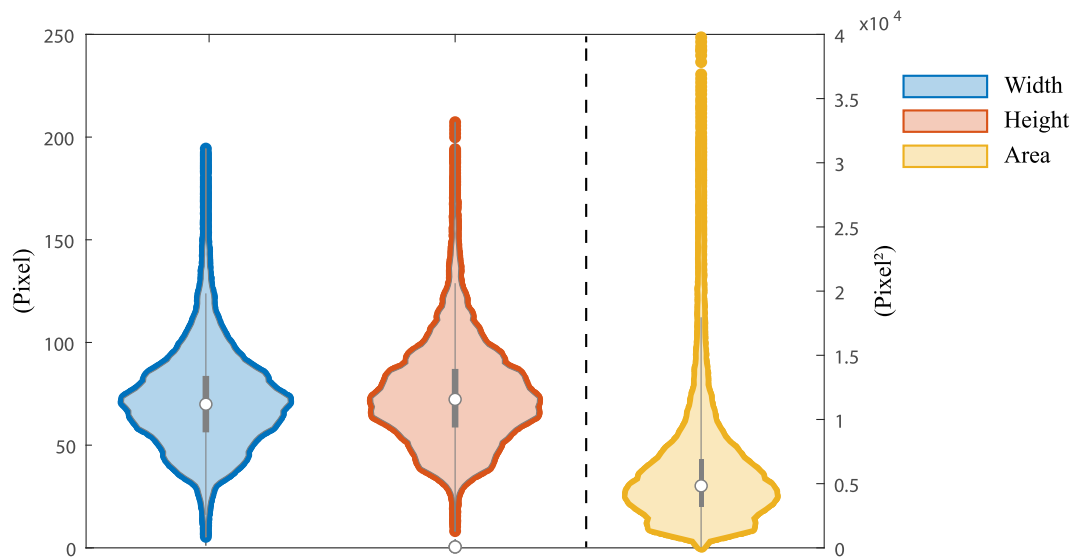


Fig. 3. Violin plot of the width, height and area of labeled apples in the database used for the sensitivity analysis. The white circles represent the median of the distribution, the thick gray line represents the interquartile range while the thin gray line shows 1.5 times the interquartile range.

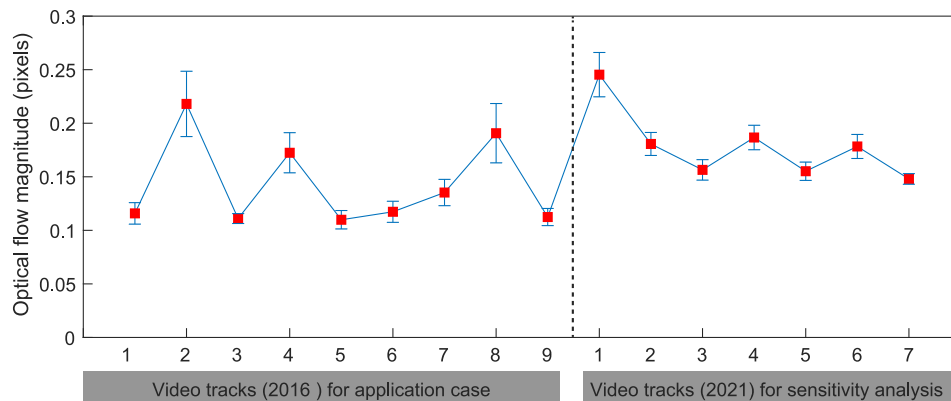


Fig. 4. Summary of the velocity in all the videos that make up the database based on the magnitude of optical flow. The mean is expressed as a red square, and the variance is depicted as a blue vertical bar.

- The WSU dataset was created by Washington State University's Agricultural Automation and Robotics Laboratory. It encompasses 2298 images of apple trees with fruit captured in different light conditions, including artificial illumination (Bhusal et al., 2019). The apples are labeled in bounding box format. In particular, this research uses the data named as “Crop Load Estimation”, which corresponds to 238 images whose image resolutions are 1494×1992 and 2976×3968 .
- The Fuji-SfM dataset was collected in Catalonia, Spain by the Research group in AgroICT & Precision Agriculture at Lleida University (Gené-Mola et al., 2020). The images were acquired with a resolution of 5184×3456 ; however, the authors extracted sub-images of 1024×1024 . In total, there are 231 sub-images and 57 images for training and validation, respectively. The ground truth is provided in polygonal format.

Table 2

Summary of the number of images that are part of the databases used to train and validate the object detectors.

Data name	Train	Validation
MinneApple	670	330
WSU	166	72
Fuji-SfM	231	57
Total	1067	459

Considering that the three public databases have different annotations formats, we converted the labels to a bounding box format. Table 2 summarizes the data distribution for the training and validation sets used with the object detectors.

3.2. Fruit detection

After defining the databases for the detection stage, we show below a brief description of the meta-architectures used in this work to detect apples. Deep learning-based strategies have outperformed classical computer vision algorithms in detection tasks, with Yolo and Faster R-CNN (and its variations) being the two widely used architectures that have achieved the highest performance in state of the art, as detailed in Sections 1 and 2.

3.2.1. Faster R-CNN

Ren et al. (2017) presented the Faster R-CNN for the first time, and applied it to detect objects in RGB images. The novelty of this detector was that the framework uses the region proposal network (RPN) to share the feature map with the detection stage. Object detection system ran at a rate of 5–17 frames per second using an NVIDIA Tesla K40 GPU. Fig. 5 shows a representation of the faster R-CNN applied to the context of apple detection. An input image passes through the feature extractor (backbone), and after a set of convolutions, a feature map is produced. The RPN slides a convolution window; a set of anchor boxes, defined by a scale and aspect ratio, are placed over each window; as a result, region proposal is obtained. Such a proposed region is filtered, considering the probability that the region contains an object. Finally, the object identification is carried out in the same manner as with the Fast-R-CNN (Koirala et al., 2019a). The anchor boxes used in the original implementation by Ren et al. (2017) are $k=9$. On the one hand, the authors used an aspect ratio of 1:1, 1:2, and 2:1, and on the other hand, the scale (box area) of 128, 256, and 512 pixels. Such anchor boxes are not defined for a particular database. Therefore, we followed the guidelines presented by Gené-Mola et al. (2019) for apple detection; specifically, we consider anchor boxes with an aspect ratio of 1:1 and a scale of 4. For more details on the architecture used in Faster R-CNN, the reader can visit the implementation provided by the Facebook research group.²

3.2.2. Yolo

The Yolo (you only look once) object detector is a single-stage framework, which outperforms the detection rate obtained with Faster R-CNN. Yolo has gone through several versions, from version 1 to version 7. The first three were developed by Redmon et al. (2016), Redmon and Farhadi (2017), and Redmon and Farhadi (2018), respectively. From YoloV4 onwards, other authors presented improvements over YoloV3. YoloV5 is used in this work, because it is the state-of-the-art detector of the Yolo family. Fig. 5 represents a schematic of the Yolo meta-architecture. In brief, YoloV5 uses as backbone Cross Stage Partial Networks (CSP) bottleneck to perform the feature extraction. The CSP reduces the computational cost by 20% while significantly outperforming the state-of-the-art detectors in the MS COCO object

detection database (Wang et al., 2020). The neck step combines the features obtained in different layers of features in the backbone. In particular, YoloV5 uses the PANet as neck, also known as a network aggregation feature. In the last step, YoloV5 performs the detection based on a deviation of predefined anchor boxes. YoloV5 has several models that are defined by the number of parameters and the size of the input. The larger the number of parameters, the better the mAP, but the longer the inference time. For this research, we used YoloV5l, which is the second-largest model, exceeded only by YoloV5x. Thus, YoloV5l has 46.5 million parameters and an input size of 640 pixels. For more details on the YoloV5l architecture, the reader can refer to the official implementation presented by Ultralytics.³

3.3. Fruit tracking and counting

We implemented five commonly used trackers to implement the visual tracking task: Kalman filter, kernelized correlation filter, multi-hypothesis tracking, simple online real-time tracking (SORT), and DeepSORT. Such trackers achieved state-of-the-art performance in people-tracking, and are currently used in agricultural applications, as detailed in Section 2.

3.3.1. Kalman filter

The Kalman filter is used to estimate the dynamic behavior of each bounding box obtained with the detector. The state vector is defined by x_k and the measurement vector z_k at time k :

$$x_k = [Cx_k, Cy_k, Vx_k, Vy_k, w_k, h_k]^T \quad (1)$$

$$z_k = [Cx_k, Cy_k, w_k, h_k]^T \quad (2)$$

where Cx , Cy are the coordinates of the center of each bounding box, and their corresponding velocities are Vx , Vy . The width and height of each bounding box are depicted by w and h , respectively. The Kalman filter is initialized with the initial conditions x_0 when a new bounding box (i.e., new apple) is detected. In the next frame, state prediction is performed. An intersection over union (IoU) cost matrix is obtained with such state predictions and new detections. Subsequently, the Hungarian algorithm associates the state prediction and the new detections. The Kalman filter is updated with the new measurements. A new tracker is initialized if the measurement is not associated with the previous trackers. If a tracker has no associations with the new detections for a certain number of frames, the tracker is removed.

3.3.2. Kernelized correlation filter

The kernelized correlation filter (KCF) was proposed by Henriques et al. (2015). This framework requires an initial input sample to train the tracker. This example consists of the image patch contained in a bounding box. Then, features are extracted from the input example based on the histogram of oriented gradients (HOG). The correlation map is obtained using the Fast Fourier Transform (FFT) on the target patch. The inverse FFT is then used to obtain a confidence map in the spatial domain. The maximum value in this confidence map predicts the new location of the bounding box. In the next frame, the patch is extracted, taking into account the prediction obtained in the previous step, and the HOG features are calculated, in addition to the frequency domain analysis. Then, the circulant matrix is applied to obtain the Gaussian correlation in the Fourier domain between the current patch and the patch in the previous frame. Subsequently, the new position of the tracked object is updated, and the model has trained again with the new position.

In the context of apple tracking, the KCF is initialized with each apple. For multi-apple tracking, one KCF tracker per apple is required. In addition, only one initial bounding box is needed to track in the

² <https://github.com/facebookresearch/detectron2>

³ <https://github.com/ultralytics/yolov5>

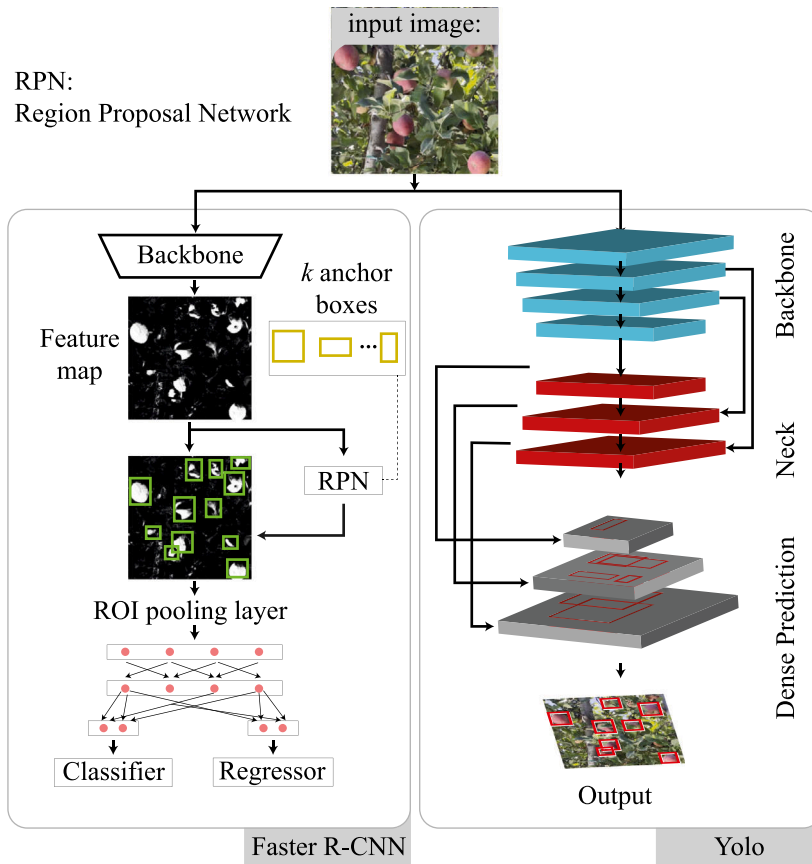


Fig. 5. Illustration of the apple detection process in the image sequence (i.e. video) using the Faster R-CNN and YoloV5 meta architectures.

video sequence. However, to exploit the new detections in each frame, the IoU and the Hungarian algorithm are applied between the new detections and the tracked apples of the previous frames to make the respective associations. Suppose there is an association between detection and tracking. In that case, the new detection is used as a training example for the KCF. Otherwise, a new tracker is initialized with each new detection. If there are no new detections, the tracker keeps the detections in the following frames. The tracker is kept until it can be updated with a new detection or removed for reaching a certain number of frames without new detections.

3.3.3. Multi hypothesis tracking

Multiple hypotheses tracking (MHT) is an algorithm for the visual tracking problem that was first proposed in 1975 (Reid, 1979) and revisited in 2015 (Kim et al., 2015). The latter approach was proposed as a tracking-by-detection framework using localized bounding boxes obtained from an object detection algorithm. The revisited MHT algorithm achieved comparable performance to state-of-the-art methods on popular tracking datasets. It is now considered a baseline in visual tracking evaluation (Kim et al., 2015).

To understand the method, some terms are defined first. Let k be the most recent frame, and M_k be the number of object detections (also called observations) in that frame k . Then, $i_k \in [0, 1, \dots, M_k]$ is the observation selected at frame k . Therefore, track hypothesis is defined as the observation sequence $i_1, i_2, \dots, i_k$, and the global hypothesis is a set of track hypotheses that do not share any measurements at any time.

The MHT approach keeps multiple hypotheses active during the entire history of a given track and stops when ambiguities of data association are resolved. Therefore, the success of MHT depends on the ability to maintain a reduced list of potential hypotheses. The

multiple hypotheses that originate from a single observation and remain active constitute a track tree. At each frame, previously existing track trees are updated by appending new observations located in the neighboring area. A new track tree is also constructed from each new single observation. Then, the best global hypothesis (those without conflict) can be found by solving a maximum weighted independent set problem. Before continuing with the next frame, the track tree is pruned by discarding those branches that deviate too much from the global hypothesis. Finally, the data association problem is solved by adopting a multi-output regularized least-squares framework (for more details, please refer to Kim et al. (2015)).

3.3.4. Simple Online Real-Time Tracking

Simple Online Real-Time Tracking (SORT) is an algorithm for tracking multiple objects in applications that require real-time execution. SORT does not include the association with appearance features to achieve fast runtime. Instead, it considers only the motion of the bounding box. This motion is modeled with the following state vector:

$$x_k = [u_k, v_k, s_k, r_k, Vu_k, Vv_k, Vs_k]^T \quad (3)$$

where: the center of a bounding box at time k is represented by its coordinates u_k, v_k ; the area s_k , the aspect ratio r_k ; the velocities Vu_k and Vv_k ; and the area velocity Vs_k . It is important to note that SORT considers the aspect ratio constant. The Kalman filter optimally predicts the position in the next frame. A data association in the subsequent frame proceeds with the correction step if there is a data association. In particular, the data association is based on the cost matrix obtained with IoU and is solved by the Hungarian algorithm. For more details on the SORT algorithm, please refer to Bewley et al. (2016).

3.3.5. DeepSORT

The DeepSORT approach includes appearance information of each target bounding box. The feature appearance is obtained with a convolutional neural network as described in Wojke et al. (2018). Similarly to SORT, the bounding boxes are described by a state vector, which has eight states:

$$x_k = [u_k, v_k, \gamma_k, h_k, Vu_k, Vv_k, V\gamma_k, Vh_k]^T \quad (4)$$

where: the center of a bounding box at time k is represented by its coordinates u_k, v_k ; the aspect ratio γ_k ; the height h_k ; and the other four states are their corresponding velocities.

DeepSORT employs the Mahalanobis distance to incorporate the motion information from the short-term Kalman filter prediction and new detections. Additionally, the cosine distance is employed to perform the association concerning appearance features in cases of long-term occlusions. The association problem is solved with a weighted sum of the Mahalanobis distance and cosine distance followed by a matching cascade. For more details, the reader is referred to Wojke et al. (2018). The deep appearance descriptor was trained using a sequence of 8 frames on average and approximately 1500 apples.

3.4. Tracker sensitivity analysis

As seen in the state-of-the-art review, the performance of trackers depends strongly on the performance (detection rate, recall or detection probability) of the detector (Bullinger et al., 2017). In this study, a sensitivity analysis is performed to evaluate the performance of the tracker as the probability of fruit detection varies. Starting from a perfect detection, that is, all the fruit are detected. Then, proceeding to eliminate certain bounding boxes gradually. Each of the bounding boxes in the ground truth is considered a random variable with uniform distribution in the interval (0,1). In other words, each bounding box at time instant k has a score between 0 and 1. Based on the score mentioned above, we select a rule for detecting or deleting each bounding box. If the apple has a score less than or equal to a threshold, that apple is considered as detected; otherwise, it is discarded. For selecting the threshold, the following values of probability of detection were considered: 0.2, 0.4, 0.6, 0.8, and 1. The threshold equal to 1 implies that all the apples are deemed to be detected in the image. In addition to considering the random elimination of detections, an analysis of the effect of downsampling the number of frames per second by a factor of 2, 3, and 6 is included, which would result in videos acquired at 15 fps, 10 fps, and 5 fps, respectively. The parameters used to tune the trackers in the 15, 10, and 5 fps cases are the same as those used for the 30 fps trackers. The performance of the tracking methods depends on the setting of these parameters. The objective of maintaining the values set for 30 fps is to analyze how resilient the tracker is to the variation of the frames per second.

3.5. Evaluation metrics for tracking

The MOTChallenge, launched in 2014, offers metrics that have become standards for evaluating single-camera multiple object trackers (Dendorfer et al., 2021). From these metrics, Multiple Object Tracking Accuracy (MOTA) was chosen to evaluate the overall performance of multiple object trackers (Dendorfer et al., 2021). MOTA includes three different sources of error in the tracking process and is defined below:

$$MOTA = 1 - \frac{\sum_k (m_k + FP_k + IDSW_k)}{\sum_k GT_k} \quad (5)$$

where k represents the frame number; GT_k is the ground truth, which is the number of objects in the entire video sequence; m_k are the missing targets, i.e., the objects that are not detected by the tracker but appear in the ground truth; FP_k are the false positives, i.e., the number of

objects detected by the tracker but not currently present in the ground-truth; and $IDSW_k$ is the number of identity switches, i.e., this metric counts the number of times the trajectory of objects changes concerning the ground truth.

To evaluate the fruit count, we introduce as a metric the relative error in each frame k as follows:

$$\varepsilon_{rel[i]} = \frac{|N_{actual[i]} - N_{est[i]}|}{N_{actual[i]}} \times 100\%, \quad (6)$$

where: N_{est} is the cumulative number of tracked fruit, determined as the number of trackers up to frame k ; and N_{actual} is the actual cumulative number of fruit up to frame k , determined by manual counting of the fruit in the images.

The relative error obtained in each video could be tabulated as the error obtained in the last frame of the sequence. However, the error could present peaks or valleys, depending on the frames at which the video sequence is stopped. As an example for the reader, Fig. 6(a) shows the evolution of the actual and estimated fruit count with a tracking algorithm. It can be seen that there is a crossover between the two curves at specific points. Such points, seen as a relative error in Fig. 6(b), present errors close to 0%. For this reason, if duration of 500 or 1000 frames is considered, the error is minimum. Due to the potential bias in the tracker's assessment, the findings provided in terms of relative error correspond to the mean of the relative error, as shown in the equation below:

$$\bar{\varepsilon}_{rel} = \frac{1}{N} \sum_{j=1}^N \varepsilon_{rel[j]} \quad (7)$$

The possible causes for finding cases like the one presented in Fig. 6 are an underestimation in the count if the detector fails to identify the fruit, mainly due to the occlusion of the leaves. On the other hand, an overestimation could be related to cases in which the fruit is visible in an instant, then occluded, and then becomes visible again so that the tracker could initialize the count of new fruit.

4. Results

This section starts by showing the sensitivity analysis of the five tracking algorithms. Next, the performance of the trackers is compared in a real application case for agriculture.

4.1. Sensitivity analysis

Fig. 7 represents the qualitative results of the trajectory followed by each of the trackers when the detection rate is varied. In particular, these results correspond to a random selection of video 4 (database 2021) and the frames between 329 and 400. According to the tracker, each line in Fig. 7 represents the trajectory followed by each apple. The trajectory is considered as the displacement of an apple with respect to the video frame. When the detection rate is 100% (i.e., ground truth), the trackers have similar behavior. However, there is a slight difference, in that the KF tends to have longer trajectories than the other trackers. At an 80% detection rate, it is noticeable that some trajectories begin to disappear, the effect being more profound in SORT. Once a detection rate of 60% is reached, the SORT tracker can practically only follow a couple of apples. Also it is to be noted that in the KCF, the number of apples tracked decreases dramatically. On the other hand, MHT and DeepSORT have similar behaviors. When the detection rate is 40%, SORT and MHT fail to track any apples and KCF has only a small set of trajectories. KF has the highest most significant number of trajectories, DeepSORT still retains a considerable number of trajectories. In the more extreme case of 20% detection rate, SORT and MHT do not track any boxes and KCF has few small trajectories. The KF tracker has more trajectories than the rest, but some of them are very small. On the other hand, DeepSORT has longer trajectories, but fewer than KF.

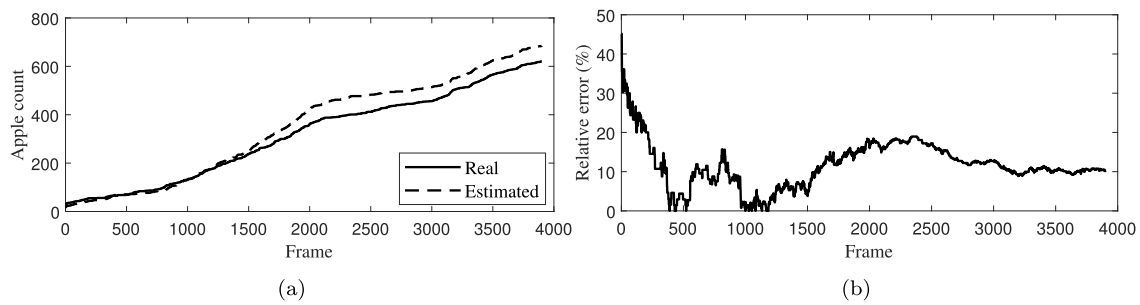


Fig. 6. Example of apple counting performance in a generic tracker. (a) ground truth and tracking method estimation and (b) absolute relative error in fruit count estimation.



Fig. 7. Qualitative evaluation of the trajectories followed by the five tracking strategies and variation of detector sensitivity. In particular, the figure represents the trajectory obtained in video 4 (database 2021) and the duration of 70 frames. The columns represent each of the trackers. The rows indicate the performance of the detector or the probability of a fruit being detected, 100% represents that the fruit is seen all the time (i.e., ground truth). Then the probability is gradually reduced in steps of 20 until detection of 20% is reached. The apple shows the final position of each fruit in the trajectories.

A quantitative analysis of the sensitivity of the trackers is presented in Fig. 8. This figure depicts the average MOTA for the seven videos discussed in Section 3.1.1. In the case of a perfect detection scenario,

MHT, SORT and DeepSORT had the best performance, 97.0%, 94.6%, and 93.0%, respectively, while KF and KCF have values below 80%. When the detection rate was reduced to 80%, MHT maintained the

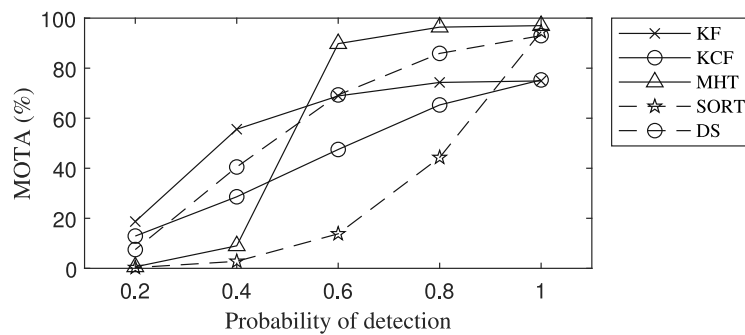


Fig. 8. Sensitivity analysis of multiple object trackers given a variation in the detection rate.

best performance (96.4%). The other trackers' performance worsened considerably, except KF, whose MOTA dropped by just 0.6%. At the next lower detection rate step, i.e., 60%, MHT maintained the best performance, followed by DeepSORT, KF, KCF, and SORT with MOTAs of 89.8%, 69.3%, 68.9%, 47.5%, and 13.8%, respectively. At the 40% detection probability, MHT's performance deteriorated dramatically, outperforming only SORT while KF, DeepSORT, and KCF had a MOTA of 55.6%, 40.5%, and 28.6%, respectively. For detection probability values of 20%, KF exceeds 18% MOTA. A sensitivity analysis extended to other evaluation metrics is presented in [Appendix](#).

The effect of reducing the video acquisition frame rate is shown in [Table 3](#). In particular, the trackers were kept under the same parameter settings in which the MOTA study was developed by having 30 frames per second. The results indicate that both the Kalman Filter and the Kernelized correlation Filter have MOTA values close to zero and even negative the more frames per second the video acquisition is reduced. On the other hand, SORT is the strategy with the best MOTA when considering 100% detection in the three downsampling cases. However, by decreasing the detection probability of the apples between 20% and 80%, MHT and DS are better when downsampling at 15 fps and 10 fps. In general, at 5 fps, all trackers have a performance close to zero and even negative values for KF and KCF. It is worth noting that when comparing the results obtained at 30 fps in [Fig. 4](#) and its counterpart at 15 fps, 10 fps and 5 fps, it is noticeable that the reduction of fps negatively affects the performance of the tracker. At a probability of detection of 80% and 30 fps, DS and MHT exceed 80% MOTA, but both trackers do not exceed 65%, 43%, and 4% when reducing fps to 15, 10, and 5, respectively. Similarly, at a probability of detection of 60%, the MOTA obtained for DS and MHT at 30 fps exceeds the MOTA obtained by the same trackers in the videos at 15 fps by approximately 15%. The difference is even more prominent at 10 fps, where the MOTA decreases by almost 40%, and when reduced to 5 fps the difference reaches practically 70% of MOTA. The probability of detection of 40% and 20% show low MOTA values using a subsampled version of the original videos, reaching values very close to zero at 5 fps for MHT, DS, and SORT.

4.2. Case study

In this case study, the effect produced by two different fruit detectors strategy based on deep learning was added to the evaluation of the trackers. In order to consider a scenario as realistic as possible, the test data were not correlated with the training/validation set. The detectors were trained with public databases; once the models were trained, detection is performed on the database presented in [Table 1](#). These detections serve as input for the tracking algorithms. Regarding the performance of the object detectors in the validation set (second column of [Table 1](#)), the results are AP@5: 75.13% and AP@5: 72.67% for Faster R-CNN and YoloV5, respectively. In contrast, the relative error in the fruit count obtained using the trackers is presented in [Fig. 9](#)

and corresponds to the meta-architectures of the YoloV5 and Faster R-CNN object detectors. The counting error for the same tracker is lower when using the Yolo detector. The lowest relative errors were obtained using the Yolo detector with the DeepSORT and KF trackers, at 20.1% and 20.5%, respectively. Considering the same trackers and the Faster R-CNN detector, the relative error for DeepSORT is 31.5%, and KF is 31.9%. The MHT tracker has an error of 32.5% and 37.1% when using Yolo and Faster R-CNN, respectively. On the other hand, the SORT tracker exceeds 50% relative counting error. Moreover, KCF exceeds 75% error when using the Yolo detector and over 90% error with Faster R-CNN.

5. Discussion

The sensitivity analysis of the trackers allows for evaluating the MOTA evolution when the probability of detection changes. In general, when considering a very optimistic scenario (i.e., perfect detection), all tracking algorithms have good MOTA results. It is important to note from [Fig. 8](#) that the SORT algorithm has a 50.3% decrease in MOTA when the probability of detection drops by 20%. This problem could be directly associated with the fact that the SORT algorithm considers the prediction with the Kalman Filter of only one instant of time. If the fruit is not tracked after this time, the tracker is removed. The prediction selection of step one is considered by the original implementation of [Bewley et al. \(2016\)](#). The authors point out that a constant velocity-based model is not a good approximation for multi-object tracking. On the other hand, MHT having a detection probability of 80% practically maintains its MOTA; and it is slightly decreased when the detection probability is 60%. This may be because MHT considers the entire detection sequence for tracking, and if detection fails between frames, the algorithm is not affected as much, i.e., it seems to be more robust to sporadic detection failures. However, when the detection probability drops to 40%, MHT's MOTA also drops, possibly because it erroneously consider shorter detection sequences.

Regarding the reduction of frames per second and the sensitivity analysis, it is evident that KF and KCF are the worst performers in all cases. This poor performance could be related to the fact that both KF and KCF depend on a tuning parameter to define the number of frames the tracker should be maintained if there are no new detections. In both cases, these parameters were heuristically tuned for the acquisition frame rate of 30 fps. Therefore, these two strategies must be adjusted considering the number of frames per second at which the videos are acquired. On the other hand, having a detection probability of 100%, the SORT tracker showed the best performance. It is important to note that SORT assumes a linear constant velocity model and is based only on the motion of the bounding boxes, so two characteristics could be considered for this tracker to have a good performance. The first is that the detector is excellent, which is complicated in applications in uncontrolled environments such as agriculture. And the second is that the camera has a relatively constant motion, as shown in [Fig. 4](#). Moreover, for probabilities of detection of 80% and below, DS and MHT

Table 3

Comparison of the trackers in terms of MOTA (%) metric when changing the acquisition frame rate. The original 30 frames per second were subsampled at 1/2, 1/3, and 1/6 of the initial sampling rate, i.e., 15 fps, 10 fps, and 5 fps, respectively. According to the sensitivity analysis, the probability of detection is expressed as 100%, 80%, 60%, 40% and 20%. The higher the MOTA then the better the tracker.

FPS	Tracker	100 (%)	80 (%)	60 (%)	40 (%)	20 (%)
15	KF	16.9	7.6	−1.9	−9.7	−7.7
	KCF	3.7	2.3	2.3	1.6	1.3
	MHT	69.5	59.4	45.4	14.3	1.3
	SORT	80.5	36.9	11.6	2.5	0.2
	DS	77.8	64.5	44.9	20.1	2.8
10	KF	−57.3	−54.9	−45.4	−31.7	−12.6
	KCF	−39.6	−28.7	−19.5	−11.4	−4.5
	MHT	56.8	42.7	28.3	7.9	0.8
	SORT	60.4	26.3	8.3	1.8	0.2
	DS	55.4	37.7	21.6	8.1	0.9
5	KF	−119.6	−90.5	−60.3	−33.3	−10.9
	KCF	−47.4	−35.3	−24.5	−14.7	−6.5
	MHT	6.9	3.8	2.1	0.7	0.1
	SORT	14.5	6.0	2.2	0.7	0.2
	DS	7.2	3.4	1.6	0.5	0.1

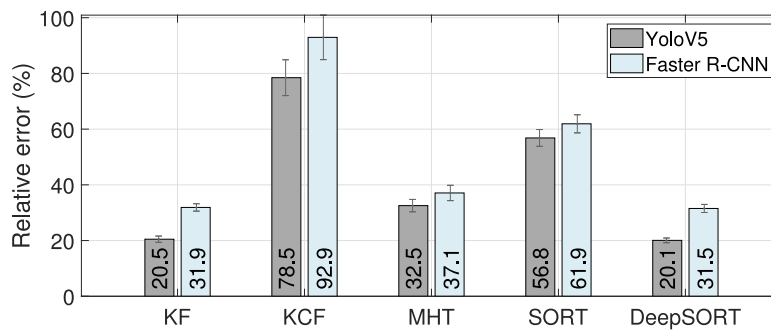


Fig. 9. Comparison of tracking strategies expressed as a relative error when using the 2016 database with YoloV5 and Faster R-CNN object detectors. The bar plot represents the mean, and the vertical line above the top of the bar is the standard deviation.

showed the best results. These results suggest that tracking based on motion and in conjunction with features extracted from the bounding boxes could be a suitable solution for apple tracking.

The Kalman Filter is configured to predict up to 8 frames or time instants. This parameter was found heuristically for the acquisition rate of 30 fps. Thus, if the fruit is not detected for a few frames, the performance should not be affected as much as shown in Fig. 8. In the KCF case, it requires only one initialization to follow the fruit through the image sequence. An absence of some bounding boxes should not significantly affect its performance. However, agricultural environments are very challenging because of changes in illumination and occlusions. Such factors can directly affect apple tracking, which is reflected in the decrease of MOTA with a linear trend. Similar results presented by Kim et al. (2017) show that KCF is not robust to occlusion and illumination changes. DeepSORT, like MHT and SORT, handles the probability of apple detection better. The results suggest that using a motion- and appearance-based tracker outperforms using motion alone as a tracking strategy. The trackers' MOTA ranking maintains without change in the probability of detection between 80% and 60%. Finally, when the detection probabilities are less than or equal to 40%, the MOTA obtained is less than 40%, except for KF, which reaches a MOTA of 55.6%. This MOTA result for the KF suggests that a constant velocity model presents a suitable approximation in data acquired in agricultural environments. In agricultural environments, a constant velocity could be achieved if the videos are acquired in a vehicle or with handed cameras with relatively constant motion.

The case study presented in Fig. 9 shows the relative error when considering two detectors as input data to the five trackers under study. The tracking errors obtained when using Faster R-CNN as a detector are

higher compared to Yolo as a detector. That situation is consistent with the results shown by Bullinger et al. (2017), which indicates that the tracker's performance depends strongly on the detection stage. Thus, it is suggestive that Yolo presented better detections for the tracker application case.

The DeepSORT tracker presents the lowest error with 20.1%, followed by KF with 20.5%. This result suggests that the constant velocity motion model employed by the Kalman filter and DeepSORT is a good approximation of the camera motion for the objects in the database obtained for this work. This constant velocity is supported by the results obtained based on the magnitude of optical flow shown in Fig. 4. However, it should be noted that in the case of the Kalman Filter, it is required to define at how many time instants the state of the enclosing box should be predicted. In particular, the prediction window was adjusted for the 30 frames per second, while in DeepSORT, all the parameters proposed by Wojke et al. (2018) were kept. By keeping the prediction window of the Kalman Filter and the same parameters in DeepSORT, it is observed that the performance in terms of MOTA is considerably reduced when reducing the frames per second of the video, as shown in Table 3. Therefore, the performance of the Kalman Filter is strongly linked to the size of the prediction window. In contrast, KCF does not use any motion model and presents the highest error among the five methods under study.

A noteworthy event in Fig. 9 is that MHT does not have an error higher than 10% than DeepSORT. Moreover, according to the sensitivity analysis, MHT is the one that presented the best results in detection probabilities greater than and equal to 60%. This result can be justified since the sensitivity analysis does not consider false positives, which affect the performance of the trackers. Further studies

Table A.1

Extension of the sensitivity analysis presented in Fig. 8. Where: Rcall, recall; Prcn, precision; GT, number of apples; FP, False Positive; FN, False Negative; IDs, identity switch.

Sensitivity	Tracker	Rcall	Prcn	GT	FP	FN	IDs	MOTA
100%	KF	93.00%	83.70%	8447	90771	35024	257	74.90%
	KCF	86.60%	88.50%	8447	56514	67088	309	75.30%
	MHT	97.30%	99.70%	8447	1426	13472	268	97.00%
	SORT	94.80%	99.80%	8447	736	26145	46	94.60%
	DS	95.50%	97.50%	8447	12453	22451	69	93.00%
80%	KF	93.30%	83.20%	8447	94469	33589	1064	74.30%
	KCF	74.70%	89.30%	8447	44939	127094	2258	65.30%
	MHT	96.90%	99.60%	8447	1939	15615	554	96.40%
	SORT	46.50%	99.90%	8447	278	268417	11006	44.30%
	DS	87.80%	98.30%	8447	7699	60998	1989	85.90%
60%	KF	89.20%	81.80%	8447	99326	54255	2338	68.90%
	KCF	55.70%	89.90%	8447	31317	222348	9581	47.50%
	MHT	90.30%	99.60%	8447	1830	48717	709	89.80%
	SORT	17.10%	99.90%	8447	59	416008	16669	13.80%
	DS	70.50%	98.80%	8447	4315	147930	1964	69.30%
40%	KF	76.80%	79.10%	8447	101697	116585	4573	55.60%
	KCF	35.60%	90.50%	8447	18794	323330	16270	28.60%
	MHT	9.20%	98.70%	8447	620	455955	191	9.00%
	SORT	3.80%	99.90%	8447	11	482706	5228	2.80%
	DS	41.10%	99.10%	8447	1895	295623	1178	40.50%
20%	KF	31.60%	73.10%	8447	58571	343063	6388	18.70%
	KCF	17.30%	91.00%	8447	8620	415121	13439	12.90%
	MHT	0.70%	95.60%	8447	158	498499	23	0.60%
	SORT	0.30%	100.00%	8447	0	500475	88	0.30%
	DS	7.60%	99.30%	8447	282	463861	148	7.50%

are needed to select an appropriate strategy for adding false positives in the sensitivity analysis.

A limitation in applying automated-based systems to fruit load estimation is the occlusion factor (OF), determined as the ratio between the total fruit on the tree and the machine vision count. The OF can vary between trees for various reasons related to canopy foliage density, including pruning history, irrigation, and nutrition (Koirala et al., 2021). Comparing the two databases in terms of OF showed that the 2016 database has an OF of 1.6129 when Yolo is used as a detector and 1.5152 using Faster R-CNN. The 2021 database has an OF of 1.2085 using Yolo and 1.1333 using Faster R-CNN. In this case, the OF was calculated as the average of the ratio of ground truth to detections at each frame in the video for each video in the database. Following this approach, the OF is related to the performance of the detectors. A lower occlusion factor is observed with the Faster R-CNN for both databases. However, it should also be considered that there is an occlusion factor during image acquisition and ground truth creation. A better approach would be considering a set of calibration trees as recommended in Anderson et al. (2021).

6. Conclusion

A set of multi-object trackers applied in a fruit-tracking agricultural application were evaluated using MOTA and counting error as performance metrics. More specifically, Kalman Filter, Kernelized Correlation Filter, Multiple Hypothesis Tracking, Simple Online RealTime, and DeepSORT were compared. For this research, two novel MOT databases for apple tracking are presented. The MHT algorithm showed the best performance (MOTA=97.00%) when ideal detection was considered. Furthermore, MHT exhibited higher robustness based on MOTA when the detection probability of the detection system was reduced down to 60%. However, in an apple tracking case study, when using MHT with Faster R-CNN and Yolo detections, the counting error was higher than KF and DeepSORT. The tracker that showed the most consistent results with respect to sensitivity and counting error tests was DeepSORT. In particular, DeepSORT obtained a MOTA of 93.00% in a perfect detection scenario and 20.10% detection error with YoloV5 as object detector. The results suggest that a constant velocity model can provide

acceptable results (as is the case with Kalman and DeepSORT). In addition, the inclusion of an appearance descriptor can reduce the counting error, as in DeepSORT. Based on the results obtained, motion- and appearance-based trackers, such as DeepSORT, seem more suitable for fruit tracking in orchard environments.

7. Supplementary material

The authors of the article acquired all videos presented in this database. We want to thank the students from Universidad Técnica Santa Maria and Universidad de los Andes who were involved in labeling the videos. The data are publicly available in the following repositories: “Case Study” (Villacrés et al., 2022a) and “Data Sensitivity” (Villacrés et al., 2022b). In case of link issues please contact the corresponding author.

CRedit authorship contribution statement

Juan Villacrés: Software, Validation, Investigation, Data curation, Writing – original draft, Visualization. **Michelle Viscaino:** Data curation, Visualization, Writing – original draft. **José Delpiano:** Investigation, Data curation, Writing – original draft. **Stavros Vougioukas:** Methodology, Formal analysis, Investigation. **Fernando Auat Cheein:** Conceptualization, Methodology, Formal analysis, Investigation, Resources, Writing – review & editing, Supervision, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

The authors would like to thank to the Advanced Center for Electrical and Electronic Engineering (AC3E), ANID Basal project FB0008 and FONDECYT grant 1201319. Authors would also like to thank to Universidad Técnica Federico Santa María, and ANID PFCHA/Doctorado Nacional/2020-21200684.

Appendix

See Table A.1.

References

- Anderson, N.T., Walsh, K.B., Koirala, A., Wang, Z., Amaral, M.H., Dickinson, G.R., Sinha, P., Robson, A.J., 2021. Estimation of fruit load in Australian mango orchards using machine vision. *Agronomy* 11 (9), 1711. <http://dx.doi.org/10.3390/agronomy11091711>.
- Bewley, A., Ge, Z., Ott, L., Ramos, F., Upcroft, B., 2016. Simple online and realtime tracking. In: *Proceedings - International Conference on Image Processing*, Vol. 2016-August. ICIP, pp. 3464–3468. <http://dx.doi.org/10.1109/ICIP.2016.7533003>.
- Bhusal, S., Karkee, M., Zhang, Q., 2019. Apple Dataset Benchmark from Orchard Environment in Modern Fruiting Wall. WSU AgRobotics Laboratory.
- Bullinger, S., Bodensteiner, C., Arens, M., 2017. Instance flow based online multiple object tracking. In: *2017 IEEE International Conference on Image Processing. ICIP, IEEE*, pp. 785–789. <http://dx.doi.org/10.1109/ICIP.2017.8296388>.
- Chen, L.-T., Lin, H.-Y., 2020. Detection, counting and maturity assessment of cherry tomatoes using multi-spectral images and machine learning techniques. In: *VISIGRAP 2020 - Proceedings of the 15th International Joint Conference on Computer Vision, Imaging and Computer Graphics Theory and Applications*, Vol. 5. pp. 759–766.
- Dendorfer, P., Ošep, A., Milan, A., Schindler, K., Cremers, D., Reid, I., Roth, S., Leal-Taixé, L., 2021. MOTChallenge: A benchmark for single-camera multiple target tracking. *Int. J. Comput. Vis.* 129 (4), 845–881. <http://dx.doi.org/10.1007/s11263-020-01393-0>.
- Gan, H., Lee, W., Alchanatis, V., Abd-Elrahman, A., 2018. An active thermography method for immature citrus fruit detection. In: *14th International Conference on Precision Agriculture*.
- Gao, F., Fang, W., Sun, X., Wu, Z., Zhao, G., Li, G., Li, R., Fu, L., Zhang, Q., 2022. A novel apple fruit detection and counting methodology based on deep learning and trunk tracking in modern orchard. *Comput. Electron. Agric.* 197, <http://dx.doi.org/10.1016/j.compag.2022.107000>.
- Gao, F., Yang, T., Fu, L., 2021. Apple fruit detection and counting based on deep learning and trunk tracking. In: *American Society of Agricultural and Biological Engineers Annual International Meeting*, Vol. 1. ASABE 2021, pp. 611–624. <http://dx.doi.org/10.13031/aim.202100193>.
- Gené-Mola, J., Sanz-Cortiel, R., Rosell-Polo, J.R., Morros, J.-R., Ruiz-Hidalgo, J., Vilaplana, V., Gregorio, E., 2020. Fuji-SfM dataset: A collection of annotated images and point clouds for fuji apple detection and location using structure-from-motion photogrammetry. *Data Brief* 30, 105591. <http://dx.doi.org/10.1016/j.dib.2020.105591>.
- Gené-Mola, J., Vilaplana, V., Rosell-Polo, J., Morros, J.-R., Ruiz-Hidalgo, J., Gregorio, E., 2019. Multi-modal deep learning for fuji apple detection using RGB-D cameras and their radiometric capabilities. *Comput. Electron. Agric.* 162, 689–698. <http://dx.doi.org/10.1016/j.compag.2019.05.016>.
- Halstead, M., McCool, C., Denman, S., Perez, T., Fookes, C., 2018. Fruit quantity and ripeness estimation using a robotic vision system. *IEEE Robot. Autom. Lett.* 3 (4), 2995–3002. <http://dx.doi.org/10.1109/LRA.2018.2849514>.
- Hamuda, E., Mc Ginley, B., Glavin, M., Jones, E., 2018. Improved image processing-based crop detection using Kalman filtering and the Hungarian algorithm. *Comput. Electron. Agric.* 148, 37–44. <http://dx.doi.org/10.1016/j.compag.2018.02.027>.
- Häni, N., Roy, P., Isler, V., 2020. A comparative study of fruit detection and counting methods for yield mapping in apple orchards. *J. Field Robotics* 37 (2), 263–282. <http://dx.doi.org/10.1002/rob.21902>.
- Hani, N., Roy, P., Isler, V., 2020. MinneApple: A benchmark dataset for apple detection and segmentation. *IEEE Robot. Autom. Lett.* 5 (2), 852–858. <http://dx.doi.org/10.1109/LRA.2020.2965061>.
- He, L., Fang, W., Zhao, G., Wu, Z., Fu, L., Li, R., Majeed, Y., Dhupia, J., 2022. Fruit yield prediction and estimation in orchards: A state-of-the-art comprehensive review for both direct and indirect methods. *Comput. Electron. Agric.* 195, <http://dx.doi.org/10.1016/j.compag.2022.106812>.
- Henriques, J., Caseiro, R., Martins, P., Batista, J., 2015. High-speed tracking with kernelized correlation filters. *IEEE Trans. Pattern Anal. Mach. Intell.* 37 (3), 583–596. <http://dx.doi.org/10.1109/TPAMI.2014.2345390>.
- Hung, C., Underwood, J., Nieto, J., Sukkarieh, S., 2015. A feature learning based approach for automated fruit yield estimation. In: *Mejias, L., Corke, P., Roberts, J. (Eds.), Field and Service Robotics: Results of the 9th International Conference. Springer International Publishing, Cham*, pp. 485–498. http://dx.doi.org/10.1007/978-3-319-07488-7_33.
- Jarvinen, T.D., Choi, D., Heinemann, P., Baugher, T.A., 2018. Multiple object tracking-by-detection for fruit counting on an apple tree canopy. p. 1.
- Kang, H., Chen, C., 2020. Fast implementation of real-time fruit detection in apple orchards using deep learning. *Comput. Electron. Agric.* 168, <http://dx.doi.org/10.1016/j.compag.2019.105108>.
- Kim, K.-Y., Kwon, J.-S., Cho, K.-S., 2017. Multi-object tracker using kemelized correlation filter based on appearance and motion model. In: *2017 19th International Conference on Advanced Communication Technology. ICACT, IEEE*, pp. 761–764. <http://dx.doi.org/10.23919/ICACT.2017.7890195>.
- Kim, C., Li, F., Ciptadi, A., Reh, J.M., 2015. Multiple hypothesis tracking revisited. In: *Proceedings of the IEEE International Conference on Computer Vision*, pp. 4696–4704. <http://dx.doi.org/10.1109/ICCV.2015.533>.
- Koirala, A., Walsh, K.B., Wang, Z., 2021. Attempting to estimate the unseen—correction for occluded fruit in tree fruit load estimation by machine vision with deep learning. *Agronomy* 11 (2), 347.
- Koirala, A., Walsh, K., Wang, Z., McCarthy, C., 2019a. Deep learning – Method overview and review of use for fruit detection and yield estimation. *Comput. Electron. Agric.* 162, 219–234. <http://dx.doi.org/10.1016/j.compag.2019.04.017>.
- Koirala, A., Walsh, K., Wang, Z., McCarthy, C., 2019b. Deep learning for real-time fruit detection and orchard fruit load estimation: Benchmarking of ‘MangoYOLO’. *Precis. Agric.* 20 (6), 1107–1135. <http://dx.doi.org/10.1007/s11119-019-09642-0>.
- Liu, X., Chen, S., Aditya, S., Sivakumar, N., Dcunha, S., Qu, C., Taylor, C., Das, J., Kumar, V., 2018. Robust fruit counting: Combining deep learning, tracking, and structure from motion. In: *IEEE International Conference on Intelligent Robots and Systems*, pp. 1045–1052. <http://dx.doi.org/10.1109/IROS.2018.8594239>.
- Liu, X., Chen, S., Liu, C., Shivakumar, S., Das, J., Taylor, C., Underwood, J., Kumar, V., 2019. Monocular camera based fruit counting and mapping with semantic data association. *IEEE Robot. Autom. Lett.* 4 (3), 2296–2303. <http://dx.doi.org/10.1109/LRA.2019.2901987>.
- Meena, N., Aggarwal, N., Joshi, S., Gupta, S., 2014. Development of detection, counting and yield estimation algorithm for agricultural products. *Int. J. Eng. Res. Technol.* 3 (7), 590–595.
- Redmon, J., Divvala, S., Girshick, R., Farhadi, A., 2016. You only look once: Unified, real-time object detection. In: *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, Vol. 2016-December. pp. 779–788. <http://dx.doi.org/10.1109/CVPR.2016.91>.
- Redmon, J., Farhadi, A., 2017. YOLO9000: Better, faster, stronger. In: *Proceedings - 30th IEEE Conference on Computer Vision and Pattern Recognition*, Vol. 2017-January. CVPR 2017, pp. 6517–6525. <http://dx.doi.org/10.1109/CVPR.2017.690>.
- Redmon, J., Farhadi, A., 2018. Yolov3: An incremental improvement. *arXiv preprint arXiv:1804.02767*.
- Reid, D., 1979. An algorithm for tracking multiple targets. *IEEE Trans. Automat. Control* 24 (6), 843–854. <http://dx.doi.org/10.1109/TAC.1979.1102177>.
- Ren, S., He, K., Girshick, R., Sun, J., 2017. Faster R-CNN: Towards real-time object detection with region proposal networks. *IEEE Trans. Pattern Anal. Mach. Intell.* 39 (6), 1137–1149. <http://dx.doi.org/10.1109/TPAMI.2016.2577031>.
- Roy, A.M., Bhaduri, J., 2022. Real-time growth stage detection model for high degree of occlusion using DenseNet-fused YOLOv4. *Comput. Electron. Agric.* 193, 106694. <http://dx.doi.org/10.1016/j.compag.2022.106694>, URL <https://www.sciencedirect.com/science/article/pii/S0168169922000114>.
- Roy, P., Dong, W., Isler, V., 2018. Registering reconstructions of the two sides of fruit tree rows. In: *IEEE International Conference on Intelligent Robots and Systems*, pp. 7697–7702. <http://dx.doi.org/10.1109/IROS.2018.8594167>.
- Stein, M., Bargoti, S., Underwood, J., 2016. Image based mango fruit detection, localisation and yield estimation using multiple view geometry. *Sensors (Switzerland)* 16 (11), <http://dx.doi.org/10.3390/s16111915>.
- Tian, Y., Yang, G., Wang, Z., Wang, H., Li, E., Liang, Z., 2019. Apple detection during different growth stages in orchards using the improved YOLO-V3 model. *Comput. Electron. Agric.* 157, 417–426. <http://dx.doi.org/10.1016/j.compag.2019.01.012>.
- Vasconez, J., Delpiano, J., Vougioukas, S., Auat Cheein, F., 2020. Comparison of convolutional neural networks in fruit detection and counting: A comprehensive evaluation. *Comput. Electron. Agric.* 173, <http://dx.doi.org/10.1016/j.compag.2020.105348>.
- Villacrés, J., Cheein, F., 2020. Detection and characterization of cherries: A deep learning usability case study in Chile. *Agronomy* 10 (6), <http://dx.doi.org/10.3390/agronomy10060835>.
- Villacrés, J., Viscaino, M., Delpiano, J., Vougioukas, S., Auat Cheein, F., 2022a. Apple orchard production estimation using deep learning strategies: a comparison of tracking-by-detection algorithms - CaseStudy [Data set]. Zenodo <http://dx.doi.org/10.5281/zenodo.7384297>.
- Villacrés, J., Viscaino, M., Delpiano, J., Vougioukas, S., Auat Cheein, F., 2022b. Apple orchard production estimation using deep learning strategies: a comparison of tracking-by-detection algorithms - SensitivityAnalysis [Data set]. Zenodo <http://dx.doi.org/10.5281/zenodo.7383338>.

- Wan, S., Goudos, S., 2020. Faster R-CNN for multi-class fruit detection using a robotic vision system. *Comput. Netw.* 168, <http://dx.doi.org/10.1016/j.comnet.2019.107036>.
- Wang, C.-Y., Mark Liao, H.-Y., Wu, Y.-H., Chen, P.-Y., Hsieh, J.-W., Yeh, I.-H., 2020. CSPNet: A new backbone that can enhance learning capability of CNN. In: *IEEE Computer Society Conference on Computer Vision and Pattern Recognition Workshops*, Vol. 2020-June. pp. 1571–1580. <http://dx.doi.org/10.1109/CVPRW50498.2020.00203>.
- Wang, Z., Walsh, K., Koirala, A., 2019. Mango fruit load estimation using a video based MangoYOLO—Kalman filter—Hungarian algorithm method. *Sensors (Switzerland)* 19 (12), <http://dx.doi.org/10.3390/s19122742>.
- Wojke, N., Bewley, A., Paulus, D., 2018. Simple online and realtime tracking with a deep association metric. In: *Proceedings - International Conference on Image Processing*, Vol. 2017-September. ICIP, pp. 3645–3649. <http://dx.doi.org/10.1109/ICIP.2017.8296962>.
- Xia, X., Chai, X., Zhang, N., Zhang, Z., Sun, Q., Sun, T., 2022. Culling double counting in sequence images for fruit yield estimation. *Agronomy* 12 (2), <http://dx.doi.org/10.3390/agronomy12020440>.
- Zhang, L.-L., He, D.-J., Song, H.-B., 2020. Automatic tracking of multi-fruit targets with oscillation under natural growth conditions. *Inf. Process. Agric.* 7 (4), 546–555. <http://dx.doi.org/10.1016/j.inpa.2019.11.004>.
- Zhang, Y., Yu, J., Chen, Y., Yang, W., Zhang, W., He, Y., 2022. Real-time strawberry detection using deep neural networks on embedded system (rtsd-net): An edge AI application. *Comput. Electron. Agric.* 192, <http://dx.doi.org/10.1016/j.compag.2021.106586>.